
Does Sentiment Analysis Improve Stock Return Prediction?

Candidate 66049
20% Contribution

Candidate 68218
20% Contribution

Candidate 65020
20% Contribution

Candidate 62553
20% Contribution

Candidate 60412
20% Contribution

Abstract

We investigate whether sentiment from a retail broker’s news feed and community forum, a channel under-explored relative to Twitter and financial newswires, improves a long-short trading strategy over a technical-indicator baseline. Using Trading 212 data on twelve US-listed equities (2021–2026), we run a controlled two-way ablation across four architectures (MLP, RNN, GRU, LSTM), holding the dataset and training protocol fixed and toggling sentiment as the only variable. Sentiment is extracted with domain-matched frozen transformers (finance-tuned DeBERTa-v3 for news, Twitter-trained RoBERTa for community posts), aggregated into daily features, and combined with technical indicators under early fusion. Predictions from a 30-seed ensemble are converted to expected-return scores over five quantile bins and used to construct daily long-short baskets under four weighting schemes. Sentiment improves classification accuracy across all four architectures. Trading gains are largest for the MLP, where sentiment lifts the Sharpe from 1.19 to 1.57 (+32%) under dynamic confidence weighting; gains for GRU and LSTM are smaller but directionally consistent. The RNN is the exception: its recurrence already extracts strong signal from technical features alone, leaving little headroom for sentiment to add. Whether sentiment translates into trading outperformance depends jointly on architecture, tuning, and weighting scheme. Broker-channel sentiment carries genuine predictive signal, but its monetisation is sensitive to downstream design choices.

1 Introduction

Investor sentiment moves prices in ways fundamentals alone cannot explain, with effects most pronounced for small, young, and hard-to-value firms (Baker and Wurgler, 2007). Recent advances in transformer-based language models have made it feasible to extract reliable sentiment signals from financial text at scale. In parallel, deep neural networks have been shown to outperform traditional machine learning models for return prediction across a range of architectures (Huang et al., 2020). The natural next question — whether combining the two halves yields better trading strategies — is the focus of this paper.

Despite emerging investigations on sentiment-based stock market prediction, the majority of research focuses on how sentiment improves prediction on a single chosen architecture. As a result, sentiment gains in the literature are often confounded with architectural and training-protocol differences, leaving it unclear which architectures genuinely benefit from sentiment under controlled conditions. This study empirically investigates two questions: whether incorporating sentiment improves return prediction and enables more profitable trading strategies compared to a technical-indicator-only baseline, and which deep learning architecture delivers the greatest gains.

Our contributions are threefold. First, we introduce a novel sentiment dataset drawn from Trading 212 news and community forums: a broker channel that remains under-explored relative to Twitter and financial news. Second, we conduct a controlled two-way ablation that isolates the contribution of sentiment across four deep learning architectures (MLP, RNN, GRU, LSTM) under a unified early-fusion framework, holding the dataset and training protocol fixed and toggling sentiment as the only variable. Third, we show that sentiment’s benefit for trading performance is conditional on architecture, tuning, and weighting scheme, with the largest gains accruing to the MLP under a dynamic confidence-weighted strategy. Throughout, we evaluate architectures along two dimensions: classification accuracy and the profitability of the resulting trading strategies.

2 Related Work

The application of sentiment analysis to financial markets has evolved alongside advances in natural language processing. Early work relied on lexicon-based methods such as bag-of-words classifiers, which struggle with negation, sarcasm, and the context-dependent polarity that pervades financial text. Transformer architectures address these limitations: domain-adapted models such as FinBERT and finance-tuned variants of DeBERTa and RoBERTa now achieve state-of-the-art performance on financial sentiment benchmarks (Mishev et al., 2020), providing reliable per-document sentiment scores that can be aggregated into model inputs.

Given the predictive power of sentiment for market movements, considerable attention has been paid to identifying which sources of sentiment matter most. Gan et al. (2020) find that news media were the dominant source in earlier periods, but that social media forums overtook it from 2016 onward. Complementing this, Duz Tan and Tas (2021) show that the predictive performance of firm-specific Twitter sentiment remains significant even after controlling for news sentiment.

The model architecture translating inputs into forecasts is itself a critical determinant of predictive performance. Huang et al. (2020) review deep learning applications in business and finance, finding that feedforward networks suit cross-sectional data, RNNs and their variants (GRU, LSTM) excel on sequential data, and CNNs handle multicollinearity well — motivating our comparison across MLP, RNN, GRU, and LSTM with a CNN depth check. Deep architectures also exhibit notable variance: training the same network with different random initialisations can produce meaningfully different results (Ganaie et al., 2022), motivating our 30-seed ensembling protocol.

How sentiment and technical indicators are combined within a deep learning model is itself a critical design choice that can materially shape predictive performance. Chen et al. (2024) employ a late-fusion architecture in which sentiment is processed through FinBERT and technical indicators through an LSTM; the resulting vectors from each pipeline are then concatenated and passed through additional dense layers. An alternative is early fusion, demonstrated by Farhan et al. (2025), which concatenates raw price features with FinBERT-derived sentiment scores before passing them through an LSTM. While prior work has shown that incorporating sentiment improves stock market prediction over technical-indicator baselines, sentiment gains in this literature are typically reported on a single chosen architecture, conflating the contribution of sentiment with the contribution of the architecture

itself. Systematic comparisons of deep learning architectures within an early-fusion framework — holding the dataset and training protocol fixed across architectures, and toggling sentiment as the only within-architecture variable — remain underexplored.

3 Data

3.1 Data Collection

Our dataset comprises two main sources: daily price history for twelve US-listed equities and Trading 212 news articles and community forum posts, spanning from January 2021 to April 2026. The twelve tickers used for this analysis are: AAPL, AMZN, BABA, BP, COIN, GME, KO, NFLX, NVDA, O, PYPL, TSLA.

Daily price history for all twelve tickers was queried from Yahoo Finance via the `yfinance` library at daily resolution, with `auto_adjust=True` to obtain dividend- and split-adjusted open, high, low, and close prices. We use unadjusted volume from the same source. Adjustment removes split- and dividend-induced discontinuities; volume is left unadjusted to preserve actual daily trading intensity

News and community posts were scraped from Trading 212’s public social-feed endpoint (<https://live.services.trading212.com/rest/social/v7/messages/get>), which provides a paginated cursor-based API. For each stock Trading 212 has a dedicated news profile populated with relevant news articles, and a community channel containing user-generated posts. Cloudflare rate limiting blocks Google Colab IP ranges, hence the scrape was run locally using the `cloudscraper` library, and documents were stored in a MongoDB Atlas collection keyed by post ID. This design enables incremental refreshes: each scrape requests only posts newer than the latest stored ID per stock, allowing efficient re-running as the test window rolls forward.

3.2 Sample Selection

The twelve tickers were chosen to balance three objectives: sector diversity, news coverage, and community volume. As Table 1 shows, community post counts vary widely across tickers, from heavy daily coverage to days with only a handful of posts, whereas news coverage is far more uniform. This decoupling separates retail attention from broader professional market attention, and tests whether sentiment remains informative across the full spectrum of coverage levels.

Table 1: Per-ticker coverage over the sample where coverage is the fraction of trading days with at least one observation. News articles are concentrated in the post-2024 subsample.

Ticker	Sector	Community Posts	Post Cov. (%)	News Articles	News Cov. (%)
AAPL	Technology	1,815	58.9	2,125	87.3
AMZN	Consumer Discretionary	1,695	53.1	2,258	86.8
BABA	Consumer Discretionary	932	39.3	608	48.9
BP	Energy	702	32.2	743	49.6
COIN	Financials	3,744	71.5	808	76.3
GME	Consumer Discretionary	9,741	56.1	154	10.1
KO	Consumer Staples	334	20.3	333	29.6
NFLX	Communication Services	1,006	33.1	810	47.0
NVDA	Technology	24,395	66.1	1,682	75.0
O	Real Estate	686	30.6	204	24.3
PYPL	Financials	898	32.8	519	38.4
TSLA	Consumer Discretionary	20,511	97.5	1,846	82.6

3.3 Sample Period and Splits

Although the raw dataset spans January 2021 – April 2026, the start of the usable sample is constrained by COIN’s April 2021 IPO and the 60-day warm-up needed to populate the longest lagged features (e.g. 60-day rolling returns). The sample is therefore split temporally into a training window [14 July 2021 – 26 April 2025] and a test window [27 April 2025 – 27 April 2026]. We employ a temporal split here to prevent the model from learning from future observations. Trading 212’s news feed only

begins producing posts in 2024, so news features are absent for roughly three of the four training years. The model effectively learns to use news features only from 2024 onwards; how this absence is encoded as feature values is detailed in Section 4.2. News features are retained because their density per post justifies inclusion, and the test set falls entirely within the news-available regime.

4 Method

4.1 Sentiment Models

To score community and news sentiment, two pretrained transformers were used to match the register of our text sources. News articles are processed with DeBERTa-v3 (184M params) (nickmuchi/deberta-v3-base-finetuned-finance-text-classification), pretrained on labelled financial text. Community posts are processed with CardiffNLP RoBERTa (125M params) (cardiffnlp/twitter-roberta-base-sentiment-latest), pretrained on tweets. We adopt this approach as a model trained on financial corpora generalises poorly to user-authored community text which is more informal and slang heavy. Both are loaded through HuggingFace’s AutoTokenizer and AutoModelForSequenceClassification and used as frozen feature extractors. At load time each model’s id2label mapping is verified or re-indexed to confirm the order of the softmax output as: negative, neutral and positive.

4.2 Feature Engineering

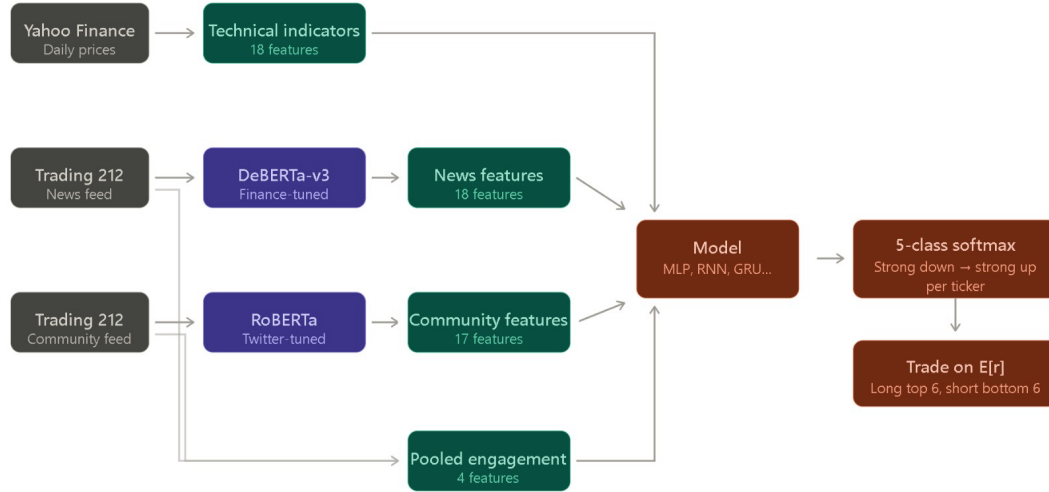


Figure 1: End-to-end pipeline for price forecasting, from data input through modeling to evaluation.

For each ticker i on each trading day t , we construct a feature vector $\mathbf{x}_{i,t-1}$ used to predict the open-to-close return $r_{i,t}$. All features are lagged by one trading day to prevent look-ahead bias: predictions for day t rely only on information available at the day- t market open. The full combined feature vector (figure 1) comprises 18 technical, 18 news-sentiment, 17 community-sentiment, and 4 pooled engagement features, totalling 57 dimensions. Univariate accuracy of every feature against the 5-class target is reported in Appendix 8.3.

Technical features (18): multi-horizon returns over $\{1, 5, 20, 60\}$ days; 5- and 20-day realized volatility and their ratio; 20-day volume ratio and 5-day volume change; intraday price range; RSI at 14- and 60-day windows; drawdown from rolling $\{5, 20, 60\}$ -day highs and recovery from same-window lows.

Per-source sentiment features. For each ticker-day with at least one post, we aggregate the per-post softmax outputs (s^{neg} , s^{neu} , s^{pos}) into a fixed-size feature vector. Posts are weighted by a

linear function of their user vote count v :

$$w(v) = \begin{cases} v & \text{if } v > 0, \\ 1 & \text{if } v = 0, \\ |v|/2 & \text{if } v < 0. \end{cases}$$

Upvotes reliably signal informative content; downvotes are ambiguous — they capture both low-quality posts and substantive minority views — so we retain them at half weight as a conservative adjustment. Zero-vote posts are kept at unit weight. Examples of high-confidence positive and negative posts under this scoring are given in Appendix 8.1.

For each source we then compute, per ticker–day: weighted means of the three sentiment dimensions, their within-day standard deviations (dispersion), and the within-day maximum negative and maximum positive scores (extremes). Weighted means are additionally expanded into rolling means over $\{5, 20, 60\}$ -day windows, giving short- and long-horizon views of the sentiment regime. All sentiment features are lagged by one trading day. To enforce look-ahead discipline, posts published after 09:30 ET are assigned to the next trading day, ensuring no feature used at prediction time reflects post-open price reactions.

This yields 17 features per source. We add one further news-only feature: the daily ratio of posts originating from top-tier publishers. Because raw sentiment levels differ systematically across tickers, each weighted-mean feature is centred on its own 252-day expanding mean.

Pooled engagement (4): four cross-source features summarising total attention. We compute pooled post counts, responses per post, and reposts per post across both sources, then express each as a ratio to its own rolling mean. Post count uses both 5- and 20-day horizons (capturing short- and medium-term activity surges), while responses and reposts use only the 20-day horizon (engagement depth being a slower-moving signal). When a source has no posts on a given ticker–day, including the entire pre-2024 period for news, counts default to zero, so pooled features collapse cleanly to whichever source is active without producing degenerate values.

4.3 Feature Sets

The models are run on two feature set splits:

- **Full (57 Features):** fed into an MLP, contains all engineered features.
- **Reduced (18 Features):** fed into an MLP as well as RNN, GRU, LSTM and CNN. Contains 6 baseline technical features (1 day return, intra-day range, RSI, volume ratio, volume ratio 20 days, volume change 5 days), 6 community features (mean and std of 3 sentiment dimensions), and same 6 features for news.

This split reflects two design considerations. First, sequential models supply temporal context through a 20 step input window, hence feeding them rolling window aggregates would partially duplicate recurrence. Second, sequence models scale poorly with input dimensionality. Training an additional MLP on the reduced set isolates the effect of recurrence: it uses the same restricted feature set as the sequence models but processes a single time step, so any performance gap between the two is attributable to temporal modeling rather than feature access. The baseline models are trained purely on the technical features adopting the same philosophy: MLP trained on full 18 technical features, while temporal models are trained on a reduced baseline of 6 technical features excluding rolling statistics.

4.4 Return Classification

For each stock $i \in \{1, \dots, N\}$ on each trading day t , let $O_{i,t}$ and $C_{i,t}$ denote the open and close prices, $r_{i,t} = C_{i,t}/O_{i,t} - 1$ the open-to-close return, $\mathbf{x}_{i,t}$ the feature vector, and f_θ the model with parameters θ . Returns are framed as five-class classification over quantile bins rather than as continuous regression, because the downstream trading decision uses only the ranking of stocks by directional conviction.

Bin edges are computed once on the training-set distribution of $r_{i,t}$ using $q = 5$ quantiles, with outermost edges replaced with $-\infty$ and $+\infty$ respectively so that any test-set return more extreme than seen in training is still classified into the strong-down or strong-up bin rather than dropped. The same

edges are applied to the test set without recomputation. Classes are labelled $\{c_{--}, c_-, c_0, c_+, c_{++}\}$ for strong-down through strong-up moves. Training-set proportions are 20% each by construction. Each return $r_{i,t}$ is mapped to a class label via these edges; see Appendix 8.2 for the resulting edges and the test-set class distribution.

The model produces a class-probability vector

$$\mathbf{p}_{i,t} = \text{softmax}(f_\theta(\mathbf{x}_{i,t})) \quad (1)$$

trained against the class label $y_{i,t}$ with cross-entropy loss. At inference the distribution collapses to a single expected-return ranking score:

$$\mathbb{E}[r_{i,t}] = -2p_{i,t}^{--} - p_{i,t}^{-} + p_{i,t}^{+} + 2p_{i,t}^{++} \quad (2)$$

where $p_{i,t}^c$ denotes the predicted probability of class c . The integer coefficients $\{-2, -1, 0, 1, 2\}$ are arbitrary equally-spaced values reflecting only the ordinal position of each class.

4.5 Ensemble Inference

To reduce seed-dependent variance, we train $M = 30$ independent copies $\{f_{\theta_m}\}_{m=1}^M$ per architecture-variant cell with seeds $\{0, \dots, M-1\}$, where θ_m denotes the parameters obtained from the m -th training run, and average the softmax outputs:

$$\bar{\mathbf{p}}_{i,t} = \frac{1}{M} \sum_{m=1}^M \text{softmax}(f_{\theta_m}(\mathbf{x}_{i,t})) \quad (3)$$

The ensemble expected-return score $\overline{\mathbb{E}[r]}_{i,t}$ is computed from $\bar{\mathbf{p}}_{i,t}$ using Eq. 2.

4.6 Trading Algorithm

On each trading day t , the ensemble scores $\overline{\mathbb{E}[r]}_{i,t}$ are used to construct a long-short portfolio with daily capital $C = \mathcal{L}120$. Positions are entered at the open and closed at end-of-day. Algorithm 1 summarises the procedure.

Algorithm 1 Daily long-short portfolio construction

Require: ensemble probabilities $\bar{\mathbf{p}}_{i,t}$ for $i = 1, \dots, N$; daily capital C

- 1: **for** each trading day t **do**
 - 2: Compute $\overline{\mathbb{E}[r]}_{i,t}$ via Eq. 2 for all i
 - 3: **if** scheme is Fixed **then**
 - 4: Long top 6 stocks by $\overline{\mathbb{E}[r]}_{i,t}$; short bottom 6
 - 5: **else**
 - 6: Long stocks with $\overline{\mathbb{E}[r]}_{i,t} > 0$; short stocks with $\overline{\mathbb{E}[r]}_{i,t} < 0$
 - 7: **end if**
 - 8: Allocate weights under scheme $\in \{\text{Fixed-Equal, Fixed-Conf, Dynamic-Equal, Dynamic-Conf}\}$
 - 9: Enter positions at open, close at end-of-day
 - 10: **end for**
 - 11: **return** daily returns, cumulative PnL
-

Fixed vs. Dynamic basket sizing. Fixed always longs the top 6 and shorts the bottom 6 by $\overline{\mathbb{E}[r]}_{i,t}$, using only the cross-sectional ranking, with capital split evenly across legs ($\mathcal{L}60$ each). Dynamic instead longs every stock with $\overline{\mathbb{E}[r]}_{i,t} > 0$ and shorts every stock with $\overline{\mathbb{E}[r]}_{i,t} < 0$; basket sizes vary day-to-day and capital is split between legs proportionally to their sizes.

Equal vs. Confidence within-leg weights. Equal divides each leg’s capital uniformly across its constituents. Confidence allocates each stock a share equal to $|\overline{\mathbb{E}[r]}_{i,t}|$ divided by the sum of $|\overline{\mathbb{E}[r]}_{j,t}|$ over the leg, concentrating capital on the highest-conviction names. Appendix 8.8 visualises the resulting per-ticker mean allocation.

5 Experiments

5.1 Model Architectures

We focus on four primary architectures (Figure 2): a multi-layer perceptron (MLP) and three recurrent models (RNN, GRU, LSTM), each producing a 5-class softmax over the quantile-binned target. The MLP receives the flat full feature set; the recurrent models receive a reduced lag-1 feature set over a 20-day look-back window, on the basis that they should learn temporal structure from the raw sequence rather than from pre-engineered rolling statistics. A smaller MLP and a 5-layer 1D CNN serve as appendix-only proof-of-concept comparisons confirming that the full MLP benefits from the rolling-window features and that the deeper CNN overfits severely (see Appendix 8.6). The reduced feature list and full architecture specifications are given in Appendix 8.4.

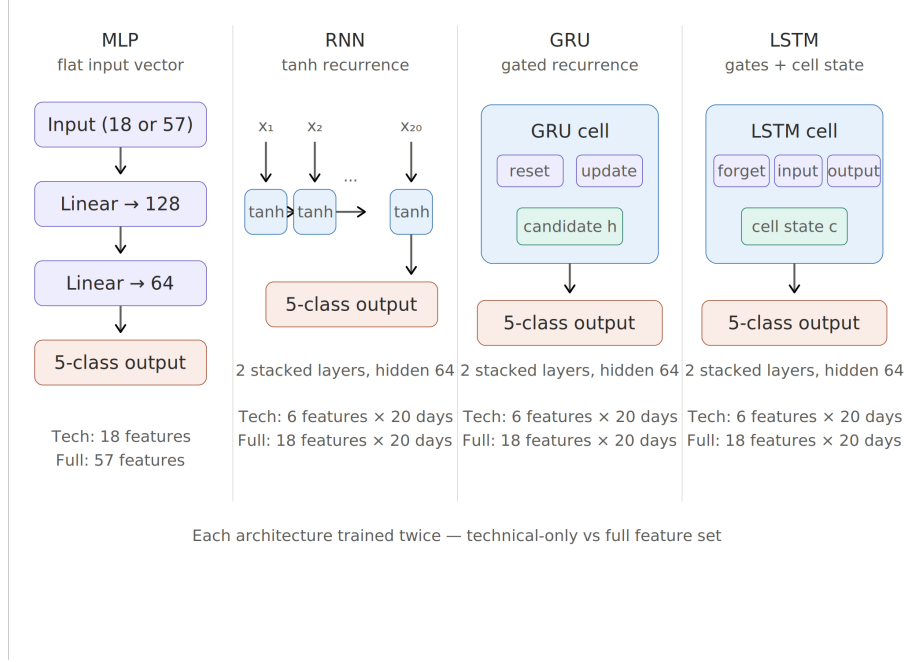


Figure 2: Model architecture specifications.

5.2 Training Protocol

All models are trained with the Adam optimiser and unweighted cross-entropy. Default hyperparameters are: learning rate 1×10^{-3} , batch size 256, dropout 0.3 (MLP) or 0.5 (recurrent), weight decay 0, and (for recurrent models) 2 layers with hidden dimension 64. Features are standardised with a `StandardScaler` fitted on the training set only. Training runs for up to 200 epochs with early stopping (patience 30) on test loss, and a `ReduceLROnPlateau` scheduler halves the learning rate after 10 epochs without improvement. The best checkpoint by test loss is restored at the end. To control seed-dependent variance, we train $M = 30$ independent runs per (architecture, variant) cell with seeds 0–29 (controlling torch, numpy, and random) and average their softmax outputs to form the ensemble.

We use unweighted cross-entropy: the five quantile classes are balanced by construction (20% each), removing the main motivation for focal or weighted alternatives. The 5-class formulation was preferred over a 3-class variant: although 3-class achieved higher raw accuracy, 5-class exceeded its 20% random baseline by a much larger relative margin and produces finer probability mass over the tails, where the $\mathbb{E}[r]$ ranking score (Equation 2) places double weight. The vote-weighting function in Section 4.2 uses linear weighting; we compared this against logarithmic weighting (preserving the same sign and zero-vote treatment) and found linear produced higher ensemble accuracy across all architectures, likely because log-compression dampens the high-engagement posts that carry the strongest retail signal.

For recurrent models, sequences are constructed per ticker (no cross-ticker contamination): each observation at time t is paired with the 19 preceding trading-day observations for the same ticker. To avoid a cold-start gap at the train-test boundary, the final 19 training rows per ticker are prepended to the test window before sequence construction.

5.3 Hyperparameter Tuning

For tuned experiments we use a two-stage Optuna procedure. The validation set is the final portion of the training window (1 September 2024 – 26 April 2025), chosen to match the temporal regime of the test set while keeping the test window (27 April 2025 – 27 April 2026) fully unseen.

Stage 1: 30 TPE-sampled trials per (architecture, variant), each evaluated on a single training seed against validation loss. *Stage 2:* the top-3 stage-1 configurations are re-evaluated across 3 seeds and ranked by mean validation loss. The two-stage design uses cheap single-seed search to scan the space and a small multi-seed sweep to filter out configurations that win on a single lucky seed but generalise poorly. The winning configuration is then retrained as the full 30-seed ensemble on the complete training window and evaluated on the test set under the identical protocol used for the untuned runs; tuned and untuned results are reported side-by-side in Section 6. Search spaces and final tuned values for each (architecture, variant) combination are reported in Appendix 8.5.

5.4 Trading Baselines

The four trading strategies and their portfolio construction are defined in Section 4 and Algorithm 1. We compare against three reference strategies: **Long All** goes long £10 on every stock each day; **Short All** goes short £10 on every stock each day; **Hindsight 6L/6S** selects the six best- and six worst-performing stocks with perfect knowledge of full test-period returns and is therefore an untradeable upper bound, included as a ceiling reference only. All trading results are reported without transaction costs or bid–ask spread.

6 Results

6.1 Classification Performance

Table 2 reports per-seed mean accuracy ($\pm$ standard deviation across 30 seeds) and ensemble accuracy for all architecture–variant combinations, before and after tuning.

Per-seed classification gains from adding sentiment are modest (0.2–0.7pp) but consistent across all four primary architectures and present in both tuned and untuned settings. The largest gain is the tuned MLP, where the combined model achieves an ensemble accuracy of 27.07% versus 25.83% for the baseline — a 1.24pp absolute improvement that amounts to a 21% relative gain in learned signal above the 20% random-guessing floor. Gains amplify under ensembling most strongly for the MLP, where seed-to-seed variation in the higher-dimensional sentiment feature space is largest. The RNN untuned baseline achieves the highest ensemble accuracy overall (27.83%), reflecting its strong capacity to extract sequential signal from technical features alone — consistent with its trading performance discussed in Section 6.2.

6.2 Trading Performance

Figure 3 reports Sharpe ratios for all (architecture $\times$ variant $\times$ tuning $\times$ strategy) combinations. CNN results are shown for completeness but, as noted in Section 4, were retained only as a depth check; the analysis below focuses on the four primary architectures.

Table 2: Classification results by architecture $\times$ variant $\times$ tuning. Mean Acc and Std Acc are computed across 30 independent seeds; Ensemble Acc is the accuracy of the 30-seed averaged softmax predictions. Within each (architecture, tuning) cell, **bold** marks the better of baseline vs. combined on Mean Acc and Ens Acc separately. Random-guessing baseline: 20.0%.

Model	Variant	Untuned			Tuned		
		Mean Acc	Std Acc	Ens Acc	Mean Acc	Std Acc	Ens Acc
MLP	baseline	0.2608	0.0052	0.2593	0.2604	0.0059	0.2583
	combined	0.2630	0.0053	0.2697	0.2673	0.0045	0.2707
RNN	baseline	0.2709	0.0032	0.2783	0.2734	0.0052	0.2757
	combined	0.2682	0.0061	0.2753	0.2720	0.0056	0.2760
GRU	baseline	0.2638	0.0041	0.2647	0.2652	0.0050	0.2707
	combined	0.2676	0.0046	0.2670	0.2681	0.0064	0.2720
LSTM	baseline	0.2619	0.0023	0.2620	0.2666	0.0051	0.2713
	combined	0.2671	0.0041	0.2690	0.2699	0.0050	0.2743

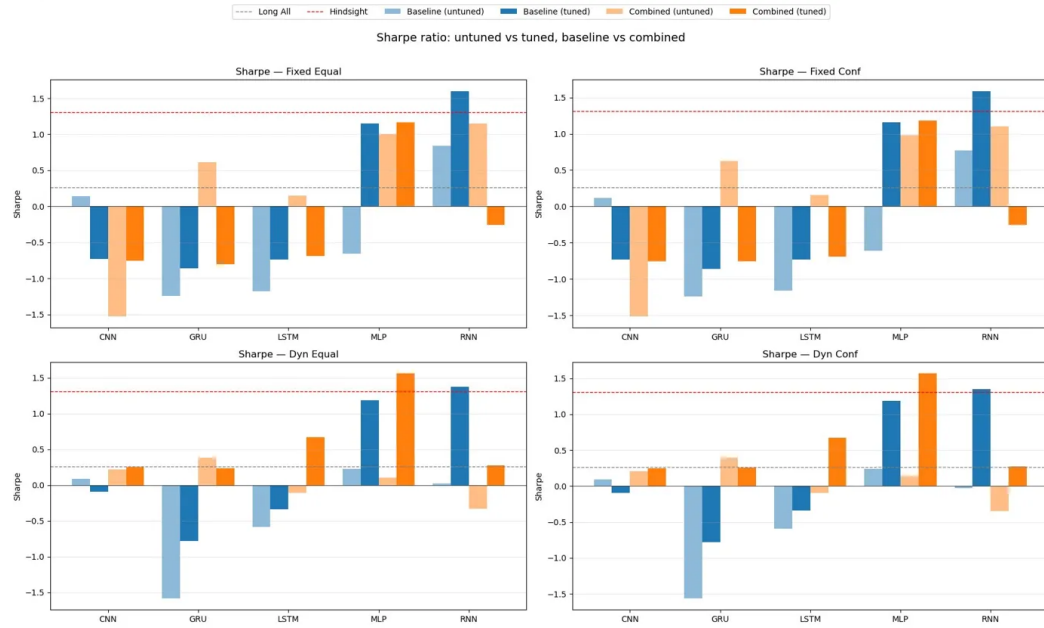


Figure 3: Sharpe ratio by architecture and variant under each trading strategy. Dashed lines show the Long All (grey) and Hindsight 6L/6S (red) reference levels.

Does sentiment help? Taking each architecture’s best strategy, all four combined variants beat Long All (0.261), whereas only two of four technical-only baselines do. Across all 32 (architecture $\times$ tuning $\times$ strategy) cells, sentiment improves Sharpe in 22 and worsens it in 10. The pattern holds beyond Sharpe: Appendix 8.7 shows sentiment also raises annualised return, reduces maximum drawdown, and lifts win rate across the untuned cells almost universally, with the same RNN-tuned exception flagged above. Sentiment helps on most risk dimensions, not just the volatility-adjusted one.

The MLP shows the clearest Sharpe gain. Under tuned Dynamic Confidence weighting, the combined MLP achieves a Sharpe of **1.570** versus **1.188** for the technical-only baseline (+32% relative), translating to 14.2% annualised return at -9.0% max drawdown. The combined MLP and the tuned RNN baseline (**1.601** under Fixed Equal) both exceed the hindsight oracle (1.308). The oracle picks a single 6L/6S basket with perfect foreknowledge of the full test period and holds it static, so a model that correctly times directional regime shifts within the year can outperform it; the comparison

should be read as contextual rather than competitive, since the oracle remains untradeable. Short All (-0.261) is beaten by every tuned model at its best strategy.

GRU and LSTM gains from sentiment are smaller but directionally consistent. The RNN is the exception: adding sentiment cuts the tuned Dynamic Confidence Sharpe from 1.353 to 0.275 — an 80% collapse, and the one architecture for which the technical-only variant is preferred. The classification result in Section 6.1 foreshadows this: the RNN untuned baseline already achieves the highest ensemble accuracy, leaving little headroom for sentiment to add.

Tuning $\times$ strategy interaction. Untuned models perform best under Fixed strategies, where the strict 6L/6S market-neutral structure compensates for poorly calibrated $\mathbb{E}[r]$ scores. Tuned models excel under Dynamic strategies, where they are trusted to size their long and short books from their own confidence.

6.3 Equity Curves

Figure 4 shows cumulative PnL over the test period (April 2025 – April 2026) for the MLP across all four strategies. Two patterns stand out. First, tuning is necessary for Dynamic strategies to work: untuned models oscillate around breakeven under Dyn Equal and Dyn Conf, while their tuned counterparts pull decisively ahead from mid-2025 onwards. Second, under Fixed strategies the tuned baseline leads, with the combined model adding little, the market-neutral 6L/6S structure is doing most of the work regardless of sentiment. The Dyn Conf panel reproduces the headline result: the tuned combined model finishes highest overall, closing at approximately £17, while the tuned baseline closes at approximately £14. Both tuned variants remain well above zero for the majority of the period, in contrast to the naive Long All and Short All benchmarks which oscillate around breakeven and finish negative respectively.



Figure 4: Cumulative PnL over the test period (April 2025 – April 2026) for the combined and technical-only MLP under Dynamic Confidence weighting.

7 Conclusion

This paper asked whether sentiment analysis improves stock return prediction in a daily long-short trading setting. Across four architectures, holding the dataset and training protocol fixed and adding only the sentiment features, the answer is yes, but the magnitude and shape of the benefit depend heavily on architecture, tuning, and strategy structure. The largest gain comes from the MLP, where

sentiment lifts the Sharpe from 1.19 to 1.57 on the Dynamic Confidence strategy — a +32% relative improvement. Among technical-only variants, the tuned RNN is strongest (Sharpe 1.59); both this and the combined MLP exceed the infeasible hindsight oracle at 1.31. The RNN is the one architecture for which adding sentiment hurts trading performance: its temporal recurrence already extracts most of the directional signal from the technical features alone.

Per-seed classification gains are modest (0.2–0.7pp) but amplify under ensembling, most strongly for the MLP. Recurrent architectures show smaller amplification, likely because the MLP receives sentiment features pre-engineered with rolling windows out to 60 days, while the recurrent models see fewer sentiment features over a 20-day window and must learn temporal structure end-to-end. Sentiment thus helps most when temporal context is supplied through feature engineering rather than left for the model to recover, and ensembling matters more for sentiment models than technical-only ones because different seeds latch onto different sentiment features.

Trading tells a more nuanced story than accuracy. Despite small classification differences, every sentiment-augmented architecture beats the naive long-all baseline (Sharpe 0.261) at its best strategy, against just two of four for the technical-only variants: models do not need to be substantially more accurate, just more accurate on the days that drive PnL. The interaction between tuning and strategy structure is striking: untuned sentiment models perform best under Fixed (market-neutral) strategies, where the 6-long/6-short structure compensates for poorly-calibrated directional predictions, while tuned models excel on Dynamic strategies, which let the model size long and short books from its own $\mathbb{E}[r]$ scores. Untuned models need market-neutral guardrails; tuned models can be trusted with directional decisions. Returning to our central questions: sentiment improves classification across all four architectures and trading for three of them, with the largest gains for the MLP.

7.1 Limitations

We identify several limitations that constrain our findings. First, news data on Trading 212 only began in 2024, meaning roughly three of the four training years contain no news features at all; the model effectively learns to use them only on the final year of training. Second, our test set is a single 12-month period (April 2025 – April 2026) characterised by lower volatility than the training era, leaving roughly 24% of test observations in the Neutral class versus the balanced 20% in training; results may not generalise to higher-volatility regimes. Third, the twelve stocks were selected partly for their active Trading 212 community presence, so sentiment may help less on names with thinner retail coverage. Fourth, we use the test set for early-stopping checkpoint selection, which would not be available in deployment and introduces some optimistic bias relative to a fully held-out evaluation. Finally, our trading evaluation does not model transaction costs or bid–ask spread, both of which would erode performance for a daily-rebalancing strategy.

7.2 Future Work

We currently employ early fusion, where each day’s posts are compressed into pooled sentiment features before being concatenated with technical indicators. A natural extension is late fusion (Chen et al., 2024), where a sentiment sub-network processes raw post embeddings directly, recovering the within-day structure our daily aggregation discards. Splitting the sentiment model into separate news-only and community-only variants would also let us attribute gains to each source individually. On the evaluation side, walk-forward backtesting across multiple test periods would address regime sensitivity, and modelling transaction costs alongside Sharpe-aware loss functions would tighten the alignment between training objective and trading performance.

Statement of Individual Contributions

Component	Contributors	Description
Code	66049, 68218	• Data scraping and MongoDB ingestion • Sentiment scoring pipeline (DeBERTa, RoBERTa) • Feature engineering and dataset construction • Model architectures, ensemble training, ablations • Backtesting, evaluation, and notebook annotation
Report	66049, 68218, 65020, 62553	• Paper drafting in NeurIPS format • Literature review and references • Figures, tables, and results analysis • Methodology and equations write-up • Editing and proofreading
Slides	60412	• Slide design and visual layout • Narrative structure and storyline • Presentation preparation

Contribution percentages: Candidate 66049: 20%, Candidate 68218: 20%, Candidate 65020: 20%, Candidate 62553: 20%, Candidate 60412: 20%.

Statement of AI Usage

Drafting and editing of the report and slides were performed by the authors. Claude’s writing assistance was limited to surface-level prose polish (typo correction, clarity edits on author-written sentences) and did not include generating content, structure, or analysis. Claude was used more substantially for portions of the code; a detailed breakdown is provided at the top of the code notebook.

References

- Baker, M. and Wurgler, J. (2007). Investor sentiment in the stock market. *Journal of Economic Perspectives*, 21(2):129–152.
- Chen, P., Boukouvalas, Z., and Corizzo, R. (2024). A deep fusion model for stock market prediction with news headlines and time series data. *Neural Computing and Applications*, 36(34):21229–21271.
- Duz Tan, S. and Tas, O. (2021). Social media sentiment in international stock returns and trading activity. *Journal of Behavioral Finance*, 22(2):221–234.
- Farhan, M., Ahmed, A., Chong, K. T., and Tayara, H. (2025). Predicting stock price direction with lstm: a fusion of financial sentiment analysis with technical indicators. *Asia-Pacific Financial Markets*.
- Gan, B., Alexeev, V., Bird, R., and Yeung, D. (2020). Sensitivity to sentiment: News vs social media. *International Review of Financial Analysis*, 67:101390.

- Ganaie, M. A., Hu, M., Malik, A. K., Tanveer, M., and Suganthan, P. N. (2022). Ensemble deep learning: A review. *Engineering Applications of Artificial Intelligence*, 115:105151.
- Huang, J., Chai, J., and Cho, S. (2020). Deep learning in finance and banking: a literature review and classification. *Frontiers of Business Research in China*, 14(1):13.
- Mishev, K., Gjorgjevikj, A., Vodenska, I., Chitkushev, L. T., and Trajanov, D. (2020). Evaluation of sentiment analysis in finance: From lexicons to transformers. *IEEE Access*, 8:131662–131682.

8 Appendix

8.1 Individual posts and sentiment scoring examples

Table 3: Representative documents and their sentiment-model outputs. News is scored by finance-DeBERTa, community by Cardiff RoBERTa-Twitter. Probabilities sum to one across (negative, neutral, positive).

Source	Ticker	Title / text (truncated)	Neg	Neu	Pos
News	BP	Grupo Santander upgrades BP to Outperform, raises PT to 5.20 GBP	0.00	0.01	0.99
News	BP	BP expects “weaker” refining margins in Q4, upstream production to fall	0.99	0.01	0.00
Community	GME	“Such a beautiful day, hopefully the candles will brighten it even more ”	0.00	0.00	0.99
Community	NVDA	“This is the worst I’ve seen fx impact ”	0.96	0.03	0.01

8.2 5 class distribution

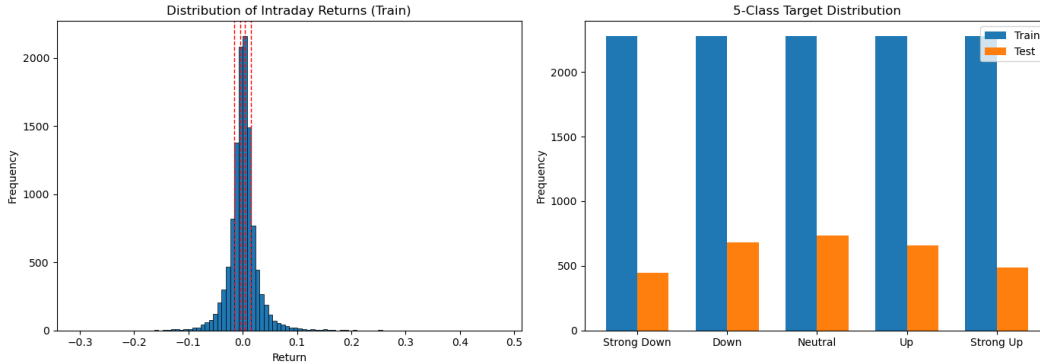


Figure 5: Training intraday return distribution with quintile bin edges (left); class distribution on train and test (right).

Bin edges (return space): $\{-0.0150, -0.0039, 0.0039, 0.0145\}$. Training class proportions are 0.20 by construction; test proportions are (0.149, 0.227, 0.244, 0.219, 0.162) for (Strong Down, Down, Neutral, Up, Strong Up), reflecting moderately reduced realised volatility in the test period.

8.3 Univariate feature performance

Test-set accuracy of a univariate multinomial logistic regression for each candidate feature on the 5-class target. Random baseline is 0.200. Rows are colour-coded by source: **technical**, **community**, **news**, **pooled engagement**.

Feature	Acc.	Feature	Acc.
vol_5d	0.2550	news_top_ratio_5d	0.2200
comm_neu_std_lag1	0.2543	news_pos_max_lag1	0.2193
intraday_range_lag1	0.2533	volume_change_5d	0.2187
comm_pos_std_lag1	0.2530	rsi_60	0.2170
drawdown_5d	0.2523	vol_ratio	0.2093
vol_20d	0.2487	news_pos_wmean_roll60	0.2090
drawdown_20d	0.2473	total_reposts_ratio	0.2067
comm_pos_max_lag1	0.2453	news_neu_wmean_roll60	0.2053
comm_neg_std_lag1	0.2433	news_neg_wmean_roll60	0.2023
comm_neu_wmean_roll20	0.2410	rsi_14	0.2020
comm_neu_wmean_roll60	0.2403	ret_60d	0.2007
recovery_60d	0.2397	news_pos_wmean_roll20	0.2003
total_count_ratio_20d	0.2393	news_neg_wmean_roll5	0.1920
comm_neg_max_lag1	0.2347	news_pos_wmean_roll5	0.1910
drawdown_60d	0.2343	news_neg_wmean_roll20	0.1903
news_neu_wmean_roll20	0.2327	comm_pos_wmean_roll20	0.1890
recovery_20d	0.2323	news_neu_wmean_lag1	0.1890
recovery_5d	0.2320	ret_20d	0.1863
news_pos_wmean_lag1	0.2320	comm_pos_wmean_roll60	0.1833
news_pos_std_lag1	0.2320	news_neu_wmean_roll5	0.1827
news_neg_max_lag1	0.2287	comm_pos_wmean_roll5	0.1813
news_neg_std_lag1	0.2287	comm_pos_wmean_lag1	0.1800
news_neg_wmean_lag1	0.2267	comm_neu_wmean_roll5	0.1773
news_neu_std_lag1	0.2263	total_responses_ratio	0.1757
volume_ratio_20d	0.2247	comm_neg_wmean_roll60	0.1740
total_count_ratio_5d	0.2230	comm_neg_wmean_roll5	0.1730
ret_5d	0.2210	comm_neg_wmean_roll20	0.1650
		comm_neg_wmean_lag1	0.1613
		comm_neu_wmean_lag1	0.1600
		ret_1d	0.1563

Table 4: Univariate logistic regression accuracy per feature, colour-coded by source.

8.4 Architecture and feature specifications

Reduced lag-1 feature set (used by recurrent models and the small MLP): `ret_1d`, `intraday_range_lag1`, `rsi_14`, `vol_ratio`, `volume_ratio_20d`, `volume_change_5d` (6 technical) plus, in the combined variant, the lag-1 weighted means and standard deviations of the three sentiment dimensions for each of community and news (12 sentiment, 18 total).

Architectures.

- **MLP**: 2 hidden layers of dimensions (128, 64) with BatchNorm, ReLU, and dropout 0.3 between layers; linear classifier to 5 logits.
- **RNN**: 2-layer `nn.RNN` (tanh) with hidden dimension 64, dropout 0.5 between layers and after the final hidden state; linear classifier on the last time-step output.
- **GRU / LSTM**: identical to RNN with `nn.GRU` / `nn.LSTM` substituted.
- **CNN1D** (depth check): five 1D convolutional layers (kernel size 5, channels 64) with BatchNorm and ReLU, followed by adaptive average pooling, dropout 0.3, and a linear classifier.
- **MLP_small** (capacity check): identical to MLP but receives the 6/18-feature reduced set, isolating the contribution of the rolling-window features.

All models output 5 logits over the quantile classes; softmax probabilities are passed to Equation 2 for the $\mathbb{E}[r]$ score.

8.5 Hyperparameter search and final values

Search space (all architectures). Learning rate: log-uniform $[10^{-4}, 10^{-2}]$. Dropout: uniform $[0.1, 0.5]$. Weight decay: log-uniform $[10^{-6}, 10^{-3}]$. Batch size: $\{128, 256, 512\}$. MLP additionally tunes `hidden1` $\in \{64, 128, 256\}$; recurrent and CNN models tune `hidden_dim` $\in \{32, 64, 128\}$. Optuna TPE sampler seed fixed to 42; stage-1 training seed fixed to 1; stage-2 refinement seeds $\{1, 2, 3\}$.

Table 5: Final tuned hyperparameters per (architecture, variant). Selected as the stage-2 winner by mean validation loss across 3 seeds, then retrained as a 30-seed ensemble on the full training window.

Arch.	Variant	LR	Dropout	Weight Decay	Batch	Hidden
MLP	baseline	5.0×10^{-3}	0.38	9.7×10^{-4}	256	64
	combined	3.2×10^{-4}	0.50	1.1×10^{-6}	256	64
RNN	baseline	2.6×10^{-3}	0.11	8.1×10^{-4}	128	128
	combined	8.7×10^{-3}	0.29	9.9×10^{-4}	128	128
GRU	baseline	8.2×10^{-3}	0.34	5.8×10^{-6}	512	32
	combined	2.9×10^{-3}	0.17	3.3×10^{-4}	128	128
LSTM	baseline	3.0×10^{-3}	0.26	6.7×10^{-5}	128	128
	combined	8.6×10^{-3}	0.10	8.1×10^{-4}	128	128
CNN	baseline	9.2×10^{-3}	0.19	5.5×10^{-4}	256	128
	combined	8.6×10^{-3}	0.10	8.1×10^{-4}	128	128

8.6 Capacity and depth checks

Table 6 shows MLP_full gains +1.0pp from sentiment on the combined variant while MLP_small gains nothing — rolling-window features are what let sentiment translate into signal. Figure 6 shows the deeper CNN overfits severely, with early-stopping firing at epochs 2–3.

Table 6: MLP capacity check. MLP_small uses the 18-feature lag-1 reduced set.

Model	Variant	Params	Avg Stop	Best Test Loss	Mean Acc	Std Acc	Ens Acc
MLP_full	baseline	11,397	4	1.5471	0.2608	0.0052	0.2593
MLP_full	combined	16,389	7	1.5499	0.2630	0.0053	0.2697
MLP_small	baseline	9,861	12	1.5556	0.2593	0.0045	0.2597
MLP_small	combined	11,397	12	1.5603	0.2508	0.0046	0.2597

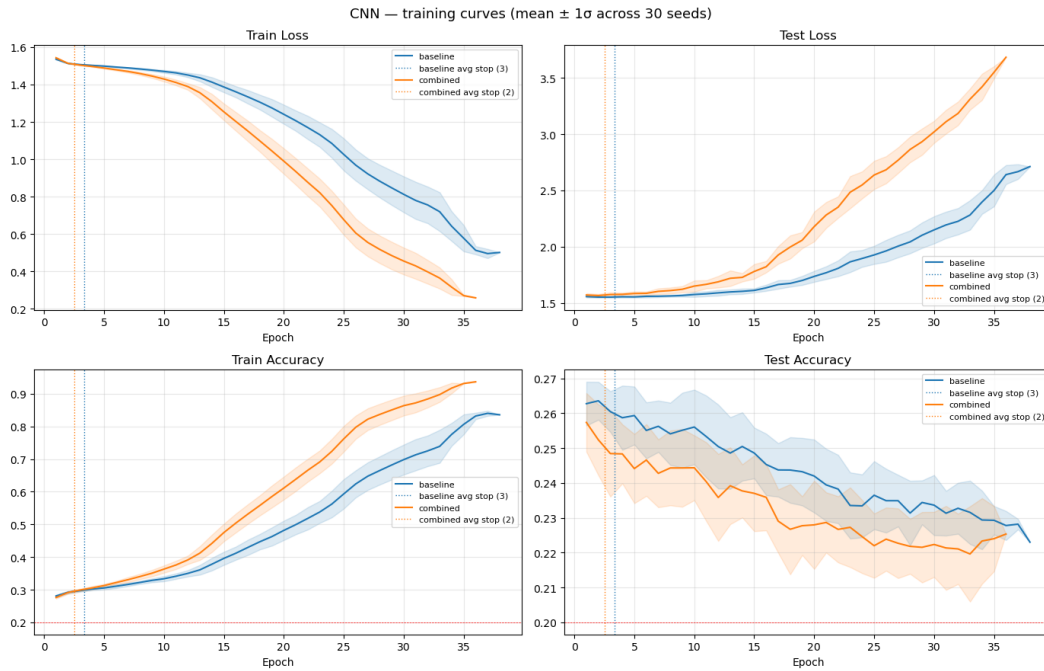


Figure 6: CNN training curves (mean $\pm 1\sigma$ across 30 seeds): severe overfitting.

8.7 Full trading metrics

Table 7 reports annualised return, maximum drawdown, and win rate for each architecture–variant cell’s best strategy by Sharpe (FE = Fixed Equal, FC = Fixed Conf, DE = Dyn Equal, DC = Dyn Conf). **Bold** marks the better of baseline vs. combined within each (architecture, tuning) block.

Table 7: Best-strategy trading metrics by architecture $\times$ variant $\times$ tuning.

Model	Variant	Untuned			Tuned		
		Ret (%)	DD (%)	Win (%)	Ret (%)	DD (%)	Win (%)
MLP	baseline	2.31 ^{DC}	-10.60 ^{DC}	51.6 ^{DC}	11.78 ^{DC}	-9.14 ^{DC}	52.0 ^{DC}
	combined	7.91^{FE}	-5.82^{FE}	49.6 ^{FE}	14.16^{DC}	-9.00 ^{DC}	55.2^{DC}
RNN	baseline	6.64 ^{FE}	-6.63 ^{FE}	52.8 ^{FE}	12.83^{FE}	-4.42^{FE}	52.0^{FE}
	combined	9.88^{FE}	-5.97^{FE}	53.6^{FE}	2.58 ^{DE}	-8.45 ^{DE}	51.6 ^{DE}
GRU	baseline	-10.18 ^{FC}	-12.46 ^{FC}	48.0 ^{FC}	-7.20 ^{DE}	-13.91 ^{DE}	46.0 ^{DE}
	combined	4.90^{FC}	-6.10^{FC}	49.6^{FC}	2.20^{DC}	-7.23^{DC}	51.6^{DC}
LSTM	baseline	-5.29 ^{DC}	-13.74 ^{DC}	47.2 ^{DC}	-2.99 ^{DC}	-8.53^{DC}	47.2 ^{DC}
	combined	1.26^{FC}	-6.40^{FC}	52.0^{FC}	8.48^{DE}	-11.61 ^{DE}	53.6^{DE}
Long All	—	3.77	-18.85	54.0	3.77	-18.85	54.0
Short All	—	-3.77	-14.00	46.0	-3.77	-14.00	46.0
Hindsight 6L/6S	—	11.77	-7.80	52.4	11.77	-7.80	52.4

8.8 Strategy allocation patterns

Stock-level conviction — mean signed £ allocation across architectures

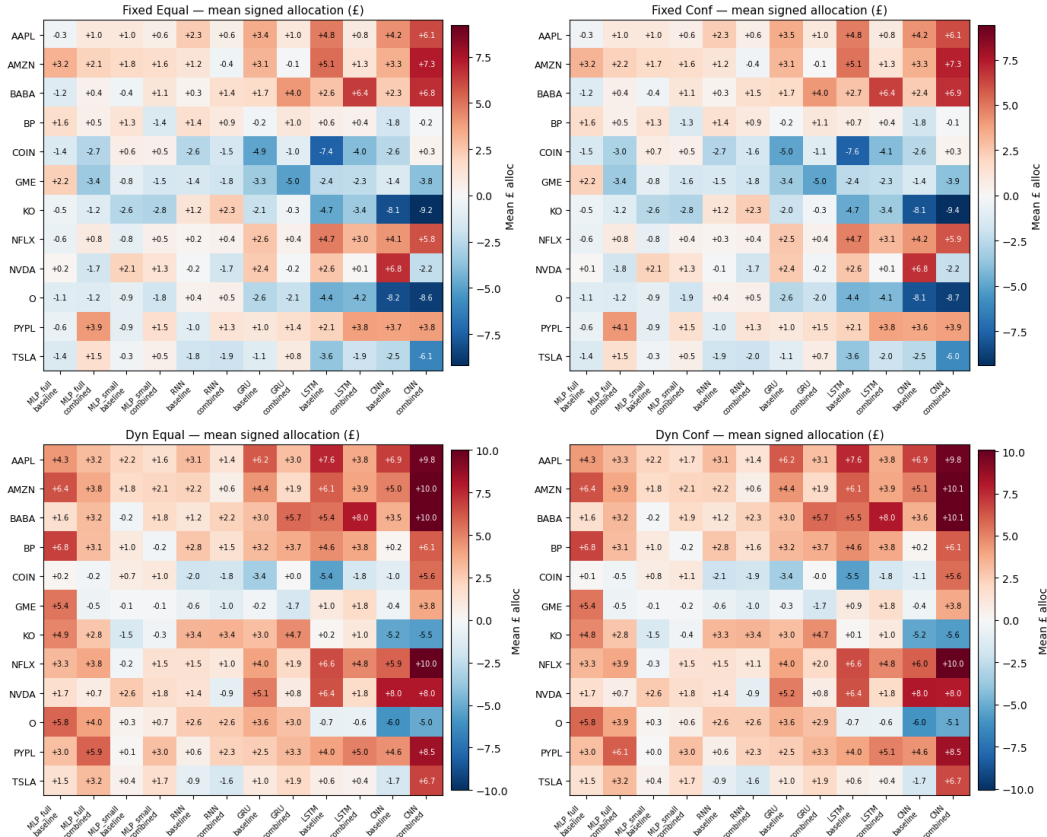


Figure 7: Mean signed daily £ allocation per ticker, by architecture–variant cell, under each of the four trading strategies. Red is net-long, blue net-short.

Three patterns are visible. Fixed Equal and Fixed Conf produce near-identical allocations, since within a top-6/bottom-6 basket the within-leg score variation is small. Dynamic strategies reach more extreme allocations than Fixed by allowing variable basket sizes, and Dyn Conf saturates further than Dyn Equal, concentrating capital on the highest-conviction names.